Nunchaku × SUNDAI CLUB

Demo, API & Tutorial

April 19,

2026

Today’s Schedule

- 10:00 Sundai intro
- 10:10 AI news
- 10:30 Nunchaku intro
- **11:00 Nunchaku technical intro — demo, API and tutorial**
- 11:30 Idea pitching and team formation
- 14:00 Check in #1 and lunch
- 16:00 Optional 5k run
- 20:00 Final presentation

Today’s Tracks

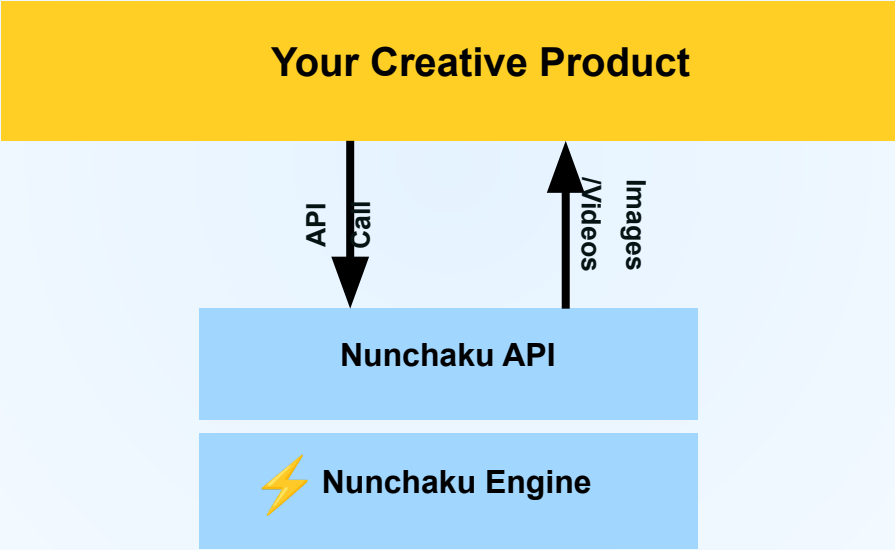
- **Track 1: Build a product using Nunchaku's fast API**
  - **If you are interested in:** product, new ideas, creative applications on top of image/video generation
  - You focus on what to build; we handle the speed

- **Track 2: Optimize a CUDA kernel from Nunchaku's inference stack**
  - **If you are interested in:** systems, hardware, performance engineering
  - Beat or match the reference benchmark

Track 1:

Build something creative on top of the world's fastest multi-modal inference API

Nunchaku supports your creativity at lightning speed!



Your Toolkits Today

- **Stable API endpoint + API keys provided today**
- **Documentation + starter repo**
- **Ultra-fast models available today**
  - **Text-to-image:**
    - Nunchaku Flux2-klein-9b (More photorealistic)
    - Nunchaku Qwen-Image: Text-to-image (Stronger at rendering text, 2 tiers)
  - **Image-editing:** Nunchaku Qwen-Image Edit (2 tiers)
  - **Text-to-video:** Nunchaku Wan2.2 t2v
  - **Image-to-video:** Nunchaku Wan2.2 i2v

Three-Step Starter Package

Step 1: Get you API key at
https://sundai.nunchaku.dev/

Step 2: Try examples in starter repo

Step 3: Start hacking!

API Reference

Parameter	Type	Description
`model`	string	`flux-1-schnell`, `flux-1-dev`
`prompt`	string	Text description
`width` / `height`	int	Image dimensions (default 1024×1024)
`num_inference_steps`	int	Quality vs. speed tradeoff
`seed`	int	Reproducibility

See README in the starter repo for full reference.

Starter Repo

```
github.com/nunchaku-ai/sundai-starter
├── python/          # raw API requests + notebook examples
├── node/            # JS/TS client
├── gradio-demo/     # instant UI wrapper
└── nextjs/          # full-stack web app scaffold
```

Track 1 Hack Ideas

- **Creative tools:** AI poster/logo maker, moodboard generator, style transfer
- **Personalization:** avatar creator, photo-to-art, custom stickers
- **Interactive:** sketch-to-image, text adventure with generated scenes
- **Productivity:** slide background generator, design asset creator
- **Wildcard:** anything that benefits from fast, high-quality image generation

Track 1 Tips

- The API is the engine. Focus your energy on product and UX
- Start with nunchaku-qwen-image — supports all speed tiers, great for iteration
- Try nunchaku-flux.2-klein-9b — distilled model, fast results
- Use tier: "radically_fast" during development — ~10x faster, 60% cheaper (Qwen only)
- Switch to tier: "fast" for your final demo
- Set a seed while iterating on prompts — reproducible results
- Chain endpoints — generate an image, edit it, then animate it
- Video takes ~30s — plan your demo flow around this
- Use response_format: "url" if you want a link instead of base64

Track 2:

Optimize a real CUDA kernel
from Nunchaku's inference stack.

Why This Matters

- Nunchaku's speed advantage comes from custom kernels
- A well-optimized kernel can be 2–10× faster than a naive implementation
- This is the actual work our engineers do
- **Today you get access to a real piece of the stack**

CUDA Mental Model

- GPU runs thousands of threads in parallel
- Threads → **warps** (32 threads) → blocks → grid
- Two regimes: memory-bound vs. compute-bound
  - Memory-bound: kernel spends most time waiting for data from HBM
  - Compute-bound: kernel spends most time doing math on the SM
- Our kernel (W4A4 GEMM with batching) is compute-bound — enough data reuse that memory is no longer the bottleneck

Memory Level	Speed	Capacity
Registers	Fastest	Per-thread, tiny
Shared memory	Fast (~20 TB/s)	Per-block, ~48–96 KB
Global memory (HBM)	Slow (~2–3 TB/s)	Entire GPU

Where Performance Hides

- **Occupancy:** enough active warps to keep the SM busy
- **Instruction-level parallelism:** overlap independent math ops
- **Tensor Core utilization:** are we feeding the Tensor Cores efficiently?
- **Quantized arithmetic:** W4A4 means 4-bit weights × 4-bit activations; dequantize at the right time

The Challenge

- **Kernels:** W4A4 GEMM (4-bit weight × 4-bit activation matrix multiply), INT4 Quantization

- Reference implementations provided:
  - Naive SIMT (Single instruction, multiple threads) is a good starting point
  - MMA (matrix multiply, accumulate) starter shows how to use Tensor Cores to increase throughput

- **Your goal:** maximize TOPs (Tera Operations Per Second) throughput for real FLUX.1 model layers

- **Our Version:** ~632 TOPs — can you match or beat it while maintaining

The Repo

```
cuda-challenge/
  GETTING_STARTED.md     # How to set up and begin
  dump_data.py           # Extracts weights & activations from FLUX
  benchmark.py           # Correctness + performance measurement
  benchmark.sh           # Entry point for benchmarking
  setup.sh               # One-time conda env setup
  reference/
    quantize.py          # Reference offline weight quantization (read-only)
    quantize_int4.cu     # Naive SIMT quantization kernel (read-only)
    gemm_int4.cu         # Naive SIMT GEMM kernel (~1 GB/s)
    gemm_int4_mma.cu     # MMA-based GEMM starting code (~63 GB/s)
    pybind.cpp           # Python bindings
  your_solution/
    quantize.py          # YOUR offline weight quantization
    kernel.cu            # YOUR CUDA kernels
    pybind.cpp           # Python bindings
```

https://github.com/nunchaku-labs/sundai-hackathon-track2

Quantization Preliminary

- **INT4 Quantization:** Storing the activations and weights with fewer bits. Replaces 16/32 bit values with bounded integers + a scale.
- Approximation: x ≈ scale * q, this introduces some error

- **W4A4**: Our weights are stored and packed to INT4 and our activations are quantized to INT4 at runtime.
- Provides a great speedup if our kernels are engineered correctly.

$$\boldsymbol{Q_X} = \text{round}\left(\frac{\boldsymbol{X}}{s_{\boldsymbol{X}}}\right), s_{\boldsymbol{X}} = \frac{\max(|\boldsymbol{X}|)}{q_{\max}}.$$

Q_x is our low-bit representaton of X, s_x is our scale. q_max = 2{k-1} = 6 for 4 bits

Quantization Preliminary

- A scale maps our floats into the INT4 range
- Too large a scale → small values round to 0
- Too small a scale → large values saturate
- **Group Size:** How many values share one scale
- Small group size: more scales, better local fit, more kernel overhead
- Large group size: fewer scales, worse fit for varying values, less kernel overhead
- Group size affects quantization error and kernel complexity

$$\boldsymbol{XW} \approx Q(\boldsymbol{X})Q(\boldsymbol{W}) = s_{\boldsymbol{X}} s_{\boldsymbol{W}} \cdot \boldsymbol{Q_X} \boldsymbol{Q_W}.$$

- Computation approximation for a linear layer with input X and weight W

How to Run & Profile

- Follow **GETTING_STARTED.md** in the repository
- Main entry point: **benchmark.sh**
  - Measures correctness and performance of your kernel

- You can use NVIDIA Nsight Compute to inspect kernel runtime, memory bandwith, tensor core utilization - all important measurements that might help

```
ncu --target-processes all \
    --set full \
    --export profile_report
\   bash benchmark.sh
```

Optimization Playbook

- **Profile first:** use NCU to confirm you're compute-bound, not memory-bound before optimizing
- **Maximize Tensor Core utilization:** check sm__pipe_tensor_op_hmma_cycles_active in ncu
- **Tune thread block shape:** drives reuse, throughput, and how well Tensor Cores are fed
- **Reduce register pressure:** spilling to local memory kills compute throughput
- **Pipeline loads and compute:** overlap dequantization with GEMM
- **AI-assisted vibe coding encouraged:** LLMs for kernel variants are fair game

AI-Assisted Optimization

- You don't have to write every optimization by hand
- Workflow: paste kernel + ncu output → ask LLM to optimize for memory coalescing
- Tools:
  - Claude / ChatGPT for code generation
  - Nsight Compute output as LLM context
  - CUDA Programming Guide §5 (memory), §6 (occupancy) for reference
  - humanize
  - autolab
- Creativity in how you use AI counts

Tips: Track 2

- Profile before you optimize — most time is wasted on the wrong thing
- One change at a time — isolate what actually helps
- Look at how cuBLAS / Triton approach similar patterns
- Ask Nunchaku engineers on-site — we'll be here all day

Get Started Now

- Track 1 API keys: [QR code / sign-up link]
- Track 2 GPU access: [SSH instructions / cloud link]
- Discord: #sundai-hack-april19
- Grab a Nunchaku team member if you're stuck

What we are interested in

	Track 1	Track 2
Technical	API integration quality	GB/s improvement over baseline
Creativity	Use case originality	Optimization approach
Demo	UX + live walkthrough	Explanation of what you tried
Impact	Real-world applicability	Insight into the kernel

We’re looking for you

- Internship / full-time conversations with the Nunchaku team
- Nunchaku swag

Go Build Something Fast

- Hack starts now
- Demos at 8pm
- Nunchaku engineers on-site all day

Thank You!

contact@nunchaku.tech